# Setup Guide

From zero to the app running on your phone, then recording the demo.

| | |
|---|---|
| **Your time** | About 60–90 minutes, one time |
| **Cost** | $0 for Supabase and Expo. About $5 of Claude API credit covers the whole demo with room to spare. |
| **What's already built** | The full app, database, security rules, AI functions and tests. You are only connecting accounts. |
| **Deadline** | Oct 26, 2026. Sources disagree on the cutoff time (12 PM, 5 PM or 8 PM ET), so confirm your district's time and that your district is participating. |

**Two rules that protect your account:** (1) The Claude API key only ever goes into Supabase secrets (Step 6). Never put it in the app, in `.env`, or in GitHub. (2) Never paste keys into a chat with any AI, including me.

## What you need first

| Account / tool | Why | Where |
|---|---|---|
| Node.js 20+ and Git on your laptop | Runs the app's dev server | nodejs.org, git-scm.com |
| Supabase account | Database, sign-in, live group sync, AI functions | supabase.com (free tier) |
| Claude Console account | The API key for screenshot reading + game plan | console.anthropic.com |
| Expo Go app on your phone (and a friend's) | Runs the app without the App Store | App Store / Google Play |
| Google Cloud project (optional) | Only for the "Continue with Google" button | console.cloud.google.com |

### 1 Get the code and install

```
git clone https://github.com/ashih-10/<your-repo>.git
cd <your-repo>
npm install
```

## 2 Create the Supabase project

1. supabase.com → **New project**. Pick a region near you and save the database password somewhere safe.
2. When it finishes, open **Project Settings → API Keys**. Copy the **Project URL** and the **publishable key** (starts with `sb_publishable_`).
3. In the code folder: copy `.env.example` to `.env` and paste both values in. The publishable key is safe in the app; the security rules protect the data.

## 3 Create the database tables and security rules

```
npx supabase login
npx supabase link --project-ref <your-project-ref>   # the ref is in your project URL
npx supabase db push
```

This runs `supabase/migrations/…_init.sql`: every table, the group rules, and row-level security. **Fallback** if the CLI gives you trouble: open **SQL Editor** in the dashboard, paste that whole file, and press Run.

## 4 Sign-in settings

1. **Authentication → Sign In / Providers → Email**: turn **off** "Confirm email" for the demo, so new accounts work immediately. (Turn it back on after the competition.)
2. **Authentication → URL Configuration → Redirect URLs**: add `exp://**` and `http://localhost:8081`. Only needed for Google sign-in.

## 5 Claude API key with a spending limit

1. Claude Console → **Billing**: add $5 of credit.
2. **Limits**: set a monthly spend limit (for example $10). This is your safety net if a bug ever loops.
3. **API Keys → Create key**. Copy it for the next step only.

What it costs: one assignment photo on Claude Haiku is roughly a tenth of a cent; one game plan on Claude Sonnet is roughly one cent. The app also caps each student at 40 AI calls a day.

## 6 Store the key on the server and deploy the AI functions

```
npx supabase secrets set ANTHROPIC_API_KEY=<paste key here>
npx supabase functions deploy extract-task
npx supabase functions deploy plan-today
```

Type the key straight into your terminal. Then clear it from your clipboard.

### 7 Run it on your phone

```
npx expo start
```

- **iPhone:** scan the QR code with the Camera app. **Android:** scan it inside Expo Go.
- Phone and laptop must be on the same Wi-Fi. School Wi-Fi often blocks this; use `npx expo start --tunnel` instead.
- Press `w` in the terminal to open the web version. That's an easy second account for the demo (groups, AI and sync all work on web; pepper *alerts* only work on phones).

### 8 Optional: "Continue with Google"

Email sign-in is enough for the demo. If you want Google: create an OAuth client (type **Web application**) in Google Cloud, add Supabase's callback URL (shown on **Authentication → Sign In / Providers → Google**) as an authorized redirect URI, then paste the client ID and secret into that Supabase page. Budget an hour; skip it if you are short on time.

### 9 Put the web version online with Vercel

> **Fastest way to see it working (about 20 minutes):** do Steps 2, 3 and 4, then this step. Steps 5 and 6 (the AI key) can come after; until then the photo and game-plan buttons show an error and everything else works.

1. vercel.com → sign up with GitHub → **Add New → Project** → import **ashih-10/Pepper**.
2. Leave the build settings alone: the repo's `vercel.json` already sets them.
3. Open **Environment Variables** and add the same two values as your `.env`: `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY`.
4. Press **Deploy**. After about 2 minutes you get a link like `pepper-xxxx.vercel.app`.
5. In Supabase, **Authentication → URL Configuration**: set **Site URL** to that link and add it to Redirect URLs.

The two values are baked into the build. If you change them later, press **Redeploy** in Vercel. Every push to GitHub redeploys automatically. Pepper alerts only work in the phone app; groups, live sync and AI all work on the web. If you see the "Almost there" screen, the environment variables are missing or misspelled.

## Recording the demo

This follows the demo script in your PRD. Before you start: on **Me**, tap **Load a demo week** and turn on **Demo clock**.

| # | Show | How |
|---|---|---|
| 1 | The problem in one line | Say it to camera: deadlines are everywhere and nothing tells you what matters today. |

| | | |
|---|---|---|
| 2 | Onboarding | Make a fresh account: add classes, pick colors, answer the goals question. |
| 3 | Photo → task | Tap the camera button, photograph a real assignment sheet, show the ⚠️ fields and the AI's question, confirm at the bottom. |
| 4 | Peppers heating up | On Today, tap **+12h** / **+1 day** on the demo clock and watch homework climb from 🫑 to 👻. |
| 5 | Groups | Second account joins with the code. The same shared test shows 🔥 Habanero for the busy student and 🌶️ Serrano for the other. Move the date and show the alert and the activity feed. |
| 6 | Milk | Check off a task. Milk pours, the pepper cools, the task is struck through. |

> **Day before recording and day before judging:** open your Supabase dashboard. Free projects pause after about a week without activity. Also tap **Remove demo tasks** on Me when you're done.

## If something goes wrong

| Symptom | Fix |
|---|---|
| "Almost there" screen | `.env` is missing or has a typo. Fix it, then restart with `npx expo start -c`. |
| Expo Go can't connect | Use `npx expo start --tunnel`. |
| Sign-up says "check your email" | Turn off "Confirm email" (Step 4). |
| "The AI had a problem" | Check the key was set (Step 6) and the Console has credit. Logs: Supabase → Edge Functions → the function → Logs. |
| "You've hit today's AI limit" | 40 calls per student per day. It resets at midnight UTC. |
| No alerts on the phone | Allow notifications for Expo Go in phone settings. On Me, tap **Send a test alert**. |
| Friend's iPhone can't open it | Make sure their Expo Go is up to date, or use the web version for the second account. |

## Running the tests

```
npm test                 # pepper formula, recurring tasks, alert planning
npx supabase start       # needs Docker Desktop
npm run test:db          # 23 security tests on a real local database
```